

项目文档

队伍名：Obug

成员：艾琮焜、符群儒、陈润隆、张义

赛道：RISC-V

项目结构：C++ & CMake

一、前端

1. 分析语法树

语法结构参照 [SysY2022语言定义](#)，使用 Flex + Bison 构成词法语法分析器。除了语言定义中定义的语法树结点，我们增加了 ConstDefList、ConstExpList 和 ConstInitValList 等 List 结点来辅助构建语法树。

```
SysY2022
├── include
│   └── common
│       └── syntax_tree.h # 语法树定义
└── src
    ├── common
    │   └── syntax_tree.c # 语法树实现
    └── parser
        ├── lexical_analyzer.c
        ├── lexical_analyzer.l # 词法规则
        ├── syntax_analyzer.c
        ├── syntax_analyzer.h
        └── syntax_analyzer.y # 语法规则
```

2. 抽象语法树

为了精炼分析语法树，并使其便于遍历输出 IR，我们在分析语法树的基础上构建了抽象语法树。

抽象语法树的特点是多态和访问者模式，我们定义了该树的访问者抽象类：抽象语法树结点负责接受访问者，由结点的实际类型决定结点如何接受访问者；访问者需要定义访问各类结点的方法，由多态的特性决定访问者如何访问结点。

SysY2022

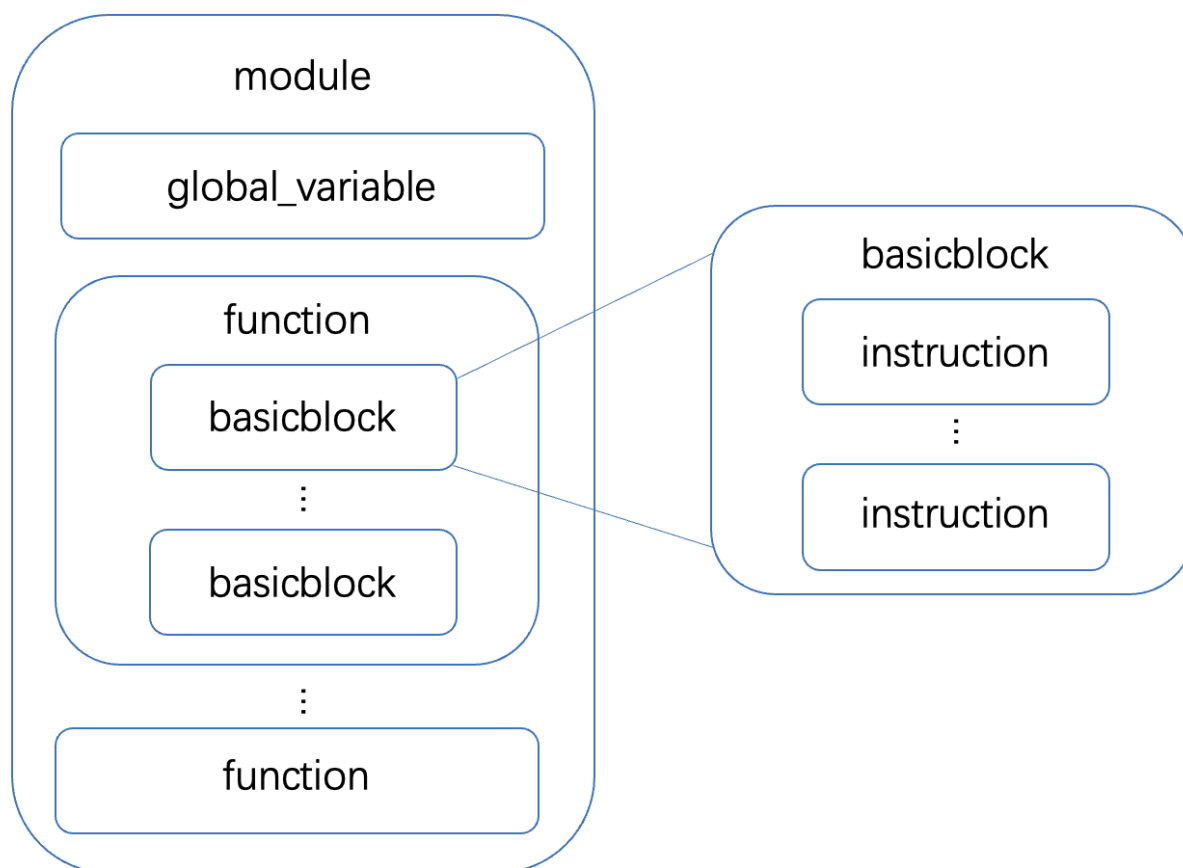
```
|— include
|   |— common
|       |— ast.hpp # 抽象语法树定义
|— src
|   |— common
|       |— ast.cpp # 抽象语法树实现
```

3. IR

我们使用 LLVM IR 作为中间代码，采用 3 地址的方式，无限寄存器 + 静态单赋值，IR 中的变量均代表了一个虚拟寄存器，并且数量无上限，每个虚拟寄存器只被赋值一次。其基本类型有 i1、i32、float 和 label，组合类型有 指针、数组和函数。

IR 的生成器即为上述提到的抽象语法树的访问者，IR 生成器通过遍历访问抽象语法树，得到构建好的 IR 结构。

IR 结构组成：



SysY2022

```
├─ include
│   └─ compiler
│       └─ SysY_builder.hpp # 访问抽象语法树，辅助构造 IR
│   └─ ir
│       └─ BasicBlock.hpp # 基本块
│       └─ Constant.hpp # 常量
│       └─ Function.hpp # 函数
│       └─ GlobalVariable.hpp # 全局变量
│       └─ ilist.hpp # 自己实现的 ilist 结构
│       └─ Instruction.hpp # 语句
│       └─ IRBuilder.hpp # IR 构造器
│       └─ IRprinter.hpp # 将 IR 模块打印为 IR 文件
│       └─ Module.hpp # IR 文件模块
│       └─ Type.hpp # IR 变量类型
│       └─ User.hpp # IR 中的左值
│       └─ Value.hpp # IR 中的右值
└─ src
    └─ compiler
        └─ SysY_builder.cpp
    └─ ir
        └─ BasicBlock.cpp
        └─ CMakeLists.txt
        └─ Constant.cpp
        └─ Function.cpp
        └─ GlobalVariable.cpp
        └─ Instruction.cpp
        └─ IRprinter.cpp
        └─ Module.cpp
        └─ Type.cpp
        └─ User.cpp
        └─ Value.cpp
```

4. MEM2REG(暂未实现)

MEM2REG 的主要目标是将内存访问转换为寄存器操作，从而提高程序的执行效率。这个优化过程主要处理程序中的局部变量，特别是那些分配在栈上的变量，将它们转换为 SSA 形式的寄存器值。

```
SysY2022
├── include
│   └── passes
│       ├── Dominators.hpp # 支配树
│       └── Mem2Reg.hpp # MEM2REG 优化
└── src
    └── passes
        ├── Dominators.cpp
        └── Mem2Reg.cpp
```

二、后端

1. IR2ASM

得到优化后的 IR 结构之后，对 IR 的每条语句进行翻译，最后将 IR 转化为栈式分配的简单 RISC-V 汇编文件。

指令类 `ASMLInstruction` 是用来描述一行汇编指令的，在 `CodeGen` 中以 `std::list` 形式组织。

寄存器分为通用寄存器 `Reg`、浮点寄存器 `FReg`。

```
SysY2022
├── include
│   └── codegen
│       ├── ASMLInstruction.hpp # RISC-V 语句定义
│       ├── CodeGen.hpp # RISC-V 代码生成器
│       ├── CodeGenUtil.hpp # RISC-V 参数
│       └── Register.hpp # RISC-V 寄存器定义
└── src
    └── codegen
        ├── CodeGen.cpp
        └── Register.cpp
```

2. 窥孔优化

对 RISC-V 汇编文件作强度削弱等优化，如乘法转换为移位、对立即数 0 和 1 的处理等。

SysY2022

```
|— include
|   |— codegen
|       |— CodeGen.hpp # 单语句强度削弱
|— src
    |— codegen
        |— CodeGen.cpp
```